

Paralelización mediante OpenMP y programación asíncrona aplicada al análisis forense de imágenes digitales

Marta López Martos and Victoria Pérez Bernabeu

Universidad de Alicante

Resumen En esta práctica se estudia la paralelización de un sistema de análisis forense de imágenes digitales mediante OpenMP y programación asíncrona en C++, orientado a sistemas multiprocesador de memoria compartida. A partir de una implementación secuencial, se identifican regiones paralelizables y se aplican técnicas de paralelismo funcional y de datos. Se analizan los tiempos de ejecución y la mejora obtenida mediante *speed-up* y eficiencia, considerando el número de hilos y el tamaño de las imágenes procesadas. Finalmente, se comparan las versiones secuencial y paralela para evaluar el impacto del paralelismo en el rendimiento del sistema.

Keywords: OpenMP · programación asíncrona · análisis forense de imágenes · C++ · rendimiento · memoria compartida

1. Introducción

1.1. Contexto del problema

El problema que se aborda en esta práctica es el análisis forense de imágenes digitales, un área que ha adquirido gran relevancia debido al aumento de manipulaciones avanzadas como los *deepfakes* o el *face swap*. Estas técnicas permiten alterar imágenes de forma muy realista, lo que hace necesario disponer de métodos capaces de detectar posibles modificaciones.

Para ello, el programa utiliza tres tipos de análisis que permiten identificar inconsistencias asociadas a manipulaciones digitales:

- **Análisis de ruido (SRM):** estudia el patrón de ruido generado por el sensor de la cámara mediante una convolución basada en el modelo *Steganalysis Rich Model*. Cuando una imagen ha sido alterada, el patrón de ruido puede presentar discontinuidades o inconsistencias detectables.
- **Análisis de nivel de error (ELA):** utiliza el comportamiento de la compresión JPEG para detectar diferencias entre la imagen original y una versión recomprimida. Las regiones modificadas suelen mostrar distintos niveles de compresión, revelando posibles manipulaciones.

- **Análisis de frecuencias (DCT):** aplica la Transformada Discreta del Coseno para analizar la distribución frecuencial de la imagen. Los algoritmos de generación artificial suelen presentar menor energía en altas frecuencias, especialmente en regiones asociadas a detalles finos.

1.2. Motivación

El principal problema de la versión inicial del sistema es que su ejecución es completamente secuencial, procesando la imagen paso a paso y ejecutando los distintos análisis de manera consecutiva ($\text{SRM} \rightarrow \text{ELA} \rightarrow \text{DCT}$). Esto limita el aprovechamiento de los recursos hardware disponibles, especialmente en sistemas multiprocesador con varios núcleos e hilos de ejecución.

Por tanto, el objetivo es transformar esta solución en una versión paralela orientada a arquitecturas de memoria compartida, donde todos los hilos puedan acceder al mismo espacio de memoria. Para ello, se emplea OpenMP para aplicar paralelismo de datos sobre operaciones intensivas, como el procesamiento de píxeles o bloques de imagen, y `std::async` para introducir paralelismo funcional entre tareas independientes.

El objetivo final es reducir el tiempo total de ejecución, maximizar el aprovechamiento de los recursos hardware disponibles y mejorar la eficiencia global del proceso de análisis forense.

1.3. Tecnologías utilizadas

Para el desarrollo de esta práctica se emplean herramientas de programación paralela en C++ orientadas a sistemas multiprocesador de memoria compartida. En este tipo de arquitecturas, todos los núcleos acceden al mismo espacio de direcciones, lo que permite implementar paralelismo mediante variables compartidas y sincronización entre hilos.

OpenMP

OpenMP es una API diseñada para implementar paralelismo de datos en arquitecturas de memoria centralizada. Su objetivo principal es dividir tareas repetitivas en subconjuntos de trabajo que puedan ejecutarse simultáneamente sobre distintos hilos.

En esta práctica, OpenMP se emplea para paralelizar los bucles más costosos del procesamiento de imágenes. Por ejemplo, al recorrer los píxeles o bloques de una imagen, las iteraciones pueden distribuirse entre varios hilos, permitiendo procesar distintas regiones de manera concurrente.

La paralelización se implementa mediante directivas como `#pragma omp parallel` y `#pragma omp for`. Además, OpenMP permite controlar el reparto de trabajo mediante la cláusula `schedule`, que define cómo se

distribuyen las iteraciones entre los hilos. En particular, el parámetro `chunk` permite especificar el tamaño de los bloques asignados a cada hilo, favoreciendo un mejor equilibrio de carga.

Otro aspecto importante es la gestión del ámbito de variables. Las cláusulas `shared` y `private` permiten indicar qué datos se comparten entre todos los hilos y cuáles deben mantenerse independientes para evitar conflictos de acceso.

`std::async`

`std::async` forma parte de la librería estándar de C++ y se utiliza para implementar paralelismo funcional mediante ejecución asíncrona. A diferencia de OpenMP, que divide una misma tarea, `std::async` permite ejecutar varias tareas independientes de forma concurrente.

En esta práctica, se utiliza para lanzar simultáneamente los algoritmos SRM, ELA y DCT, permitiendo que cada uno se ejecute en paralelo con los demás. Para garantizar que cada tarea se ejecute realmente en un hilo independiente, se utiliza la política `std::launch::async`.

La ejecución asíncrona se gestiona mediante objetos `std::future`, que permiten recuperar el resultado mediante `get()` y sincronizar la ejecución utilizando `wait()`. Esto simplifica considerablemente la gestión manual de hilos en comparación con `std::thread`.

En el entorno Linux utilizado, el uso de estas funcionalidades requiere enlazar la aplicación con la librería POSIX Threads mediante el *flag* `-lpthread` durante la compilación.

En conjunto, OpenMP y `std::async` permiten combinar paralelismo de datos y paralelismo funcional, aprovechando tanto la concurrencia interna de los algoritmos como la ejecución simultánea de tareas independientes.

2. Objetivos

El objetivo principal de esta práctica es aprender a paralelizar una aplicación en una arquitectura de memoria compartida, donde todos los núcleos acceden al mismo espacio de memoria. Para ello, se utilizan hilos de ejecución y mecanismos de sincronización orientados a mejorar el rendimiento computacional.

Los objetivos específicos son los siguientes:

- Aprender a utilizar OpenMP para implementar paralelismo de datos sobre bucles de procesamiento intensivo.
- Comprender el funcionamiento de la programación asíncrona en C++ mediante `std::async` y `std::future`.
- Transformar una aplicación secuencial de análisis forense en una versión paralela.

- Aplicar paralelismo funcional entre algoritmos independientes como SRM, ELA y DCT.
- Identificar las regiones del código susceptibles de paralelización mediante el estudio de dependencias.
- Representar la estructura de ejecución mediante diagramas de flujo y grafos de dependencias.
- Evaluar el impacto del paralelismo midiendo tiempos de ejecución, *speed-up* y eficiencia.
- Analizar la escalabilidad del programa variando el número de hilos y la talla del problema.
- Caracterizar el hardware utilizado, identificando el número de núcleos, hilos, cachés y tecnologías como Hyper-Threading.
- Comprender cómo influye la arquitectura del sistema en el rendimiento de la paralelización.
- Documentar el proceso completo de diseño, implementación y evaluación dentro de una memoria técnica estructurada.

3. Materiales y metodología

3.1. Entorno de trabajo

La práctica se desarrolló en un entorno Linux (Ubuntu) utilizando el compilador `g++`. Se emplearon las librerías `libjpeg` y `libpng` para el procesamiento y manipulación de imágenes, así como OpenMP y la librería estándar de C++ para programación asíncrona mediante `std::async`. La compilación del proyecto se gestionó mediante CMake, integrando los *flags* `-fopenmp` y `-lpthread`, siendo este último necesario para la ejecución de tareas asíncronas en entornos Linux.

El desarrollo se realizó sobre una arquitectura multiprocesador de memoria compartida, donde todos los núcleos acceden al mismo espacio de direcciones. Este modelo resulta adecuado para el uso de OpenMP y programación basada en hilos, permitiendo implementar paralelismo mediante variables compartidas.

El control de versiones y la colaboración entre los integrantes del grupo se realizaron mediante GitHub. Para la caracterización del hardware utilizado, se emplearon las herramientas `lscpu` y `cat /proc/cpuinfo`, analizando parámetros como el número de núcleos, la jerarquía de caché y la presencia de tecnologías como *Hyper-Threading*.

3.2. Procedimiento

El proceso comenzó con un análisis exhaustivo de la solución secuencial para identificar procesos críticos como SRM, ELA y DCT. Se estudió la estructura de cada algoritmo para localizar regiones computacionalmente intensivas y determinar qué partes podían ejecutarse de forma concurrente.

Posteriormente, se elaboraron diagramas de dependencias con el objetivo de identificar posibles regiones paralelizables. Este análisis permitió comprobar que los algoritmos principales podían ejecutarse de forma independiente, facilitando la aplicación de paralelismo funcional.

La implementación paralela combinó dos estrategias principales:

- **Paralelismo de datos:** aplicado mediante OpenMP sobre los bucles de procesamiento intensivo, utilizando directivas como `#pragma omp parallel for` y gestionando el reparto de iteraciones mediante políticas `schedule`.
- **Paralelismo funcional:** implementado mediante `std::async` con la política `std::launch::async`, permitiendo ejecutar simultáneamente los análisis SRM, ELA y DCT.

Finalmente, se realizaron pruebas de rendimiento para medir el *speed-up* y la eficiencia, evaluando la influencia del número de hilos y de la resolución de las imágenes sobre el comportamiento del sistema. Además, se representó la estructura del programa mediante grafos de dependencias y control de flujo.

4. Desarrollo de la práctica

4.1. Entrenamiento previo

Antes de abordar la paralelización del sistema de análisis forense, se realizaron una serie de ejercicios de entrenamiento orientados a comprender el funcionamiento de OpenMP, `std::async` y `std::vector`. Estos ejercicios permitieron familiarizarse con conceptos clave relacionados con la programación paralela y el rendimiento computacional.

4.1.1. OpenMP

El programa se ejecutó con el objetivo de analizar cómo OpenMP distribuye automáticamente las iteraciones de un bucle entre varios hilos, permitiendo paralelizar operaciones repetitivas de forma sencilla.

Uso de la variable `chunk`

En el código, la variable `chunk` se inicializa con el valor de `CHUNKSIZE` (100). Esta variable se utiliza en la cláusula `schedule` para indicar el tamaño de los bloques de iteraciones que se reparten entre los hilos.

En este caso, el bucle tiene 1000 iteraciones y se divide en bloques de 100. Cada hilo procesa un bloque y, cuando termina, puede coger otro. Esto permite equilibrar mejor la carga entre hilos, especialmente cuando las iteraciones presentan tiempos de ejecución diferentes.

Explicación del pragma `#pragma omp parallel`

La directiva `#pragma omp parallel` indica que el código que aparece a continuación se va a ejecutar en paralelo utilizando varios hilos.

En este programa se utilizan además dos cláusulas importantes:

- `shared(a, b, c, chunk)`: indica que los vectores y la variable `chunk` son compartidos entre todos los hilos. Es decir, todos acceden a los mismos datos en memoria.
- `private(i)`: la variable `i` se define como privada, lo que significa que cada hilo tiene su propia copia. Esto evita condiciones de carrera, ya que cada hilo modifica su propia copia de la variable de control del bucle.

Planificación (`schedule`) en OpenMP

La cláusula `schedule` se utiliza para decidir cómo se reparten las iteraciones del bucle entre los hilos.

En este caso se utiliza `schedule(dynamic, chunk)`, lo que significa que el reparto es dinámico: los hilos van cogiendo bloques de tamaño `chunk` conforme van terminando su trabajo. Esto es útil cuando algunas partes tardan más que otras.

También existen otras formas de planificación:

- `static`: reparto fijo desde el principio.
- `guided`: parecido al dinámico, pero los bloques se van haciendo más pequeños.
- `runtime`: se decide en tiempo de ejecución.
- `auto`: el sistema decide automáticamente.

Medidas de rendimiento en OpenMP

Para evaluar el rendimiento del programa paralelo se utilizan distintas métricas:

- Tiempo de ejecución: se mide mediante `omp_get_wtime()`, que devuelve el tiempo real transcurrido.
- *Speed-up*: compara el tiempo de ejecución secuencial frente al paralelo.
- Eficiencia: mide el grado de aprovechamiento de los hilos disponibles.
- Escalabilidad: analiza cómo varía el rendimiento al aumentar el número de hilos o la talla del problema.

4.1.2. `std::async`

En este apartado se analiza el uso de la programación asíncrona en C++ utilizando `std::async`, a partir del ejemplo proporcionado en la práctica donde se ejecutan varias tareas con diferentes tiempos de espera de forma concurrente.

El objetivo de este entrenamiento es comprender cómo funciona la ejecución asíncrona y qué diferencias existen entre las distintas políticas de lanzamiento disponibles.

Análisis de las políticas de ejecución

Se compararon las políticas `std::launch::async` y `std::launch::deferred` para analizar el comportamiento del sistema ante distintos modos de ejecución.



Figura 1. Capturas de ejecución utilizando las políticas `std::launch::async` y `std::launch::deferred`.

Para ello, se modificó el código sustituyendo `std::launch::async` por `std::launch::deferred` en la creación de las tareas:

```
std::future<int> task1 =
    std::async(std::launch::async, task, 1, 2000);
std::future<int> task2 =
    std::async(std::launch::async, task, 2, 3000);

std::future<int> task1 =
    std::async(std::launch::deferred, task, 1, 2000);
std::future<int> task2 =
    std::async(std::launch::deferred, task, 2, 3000);
```

Con `std::launch::async`, el tiempo total de ejecución es aproximadamente de 3000 ms, correspondiente a la tarea más larga. Esto indica que ambas tareas se ejecutan simultáneamente en distintos hilos.

En cambio, utilizando `std::launch::deferred`, el tiempo total se aproxima a 5000 ms, resultado de ejecutar ambas tareas de forma secuencial.

Comparación entre `async` y `deferred`

La diferencia principal entre ambas políticas reside en el momento en el que comienza la ejecución de la tarea:

- `std::launch::async`: obliga a ejecutar la tarea inmediatamente en un hilo independiente. Esto permite paralelismo real y aprovechamiento de varios núcleos.
- `std::launch::deferred`: retrasa la ejecución hasta que se solicita el resultado mediante `get()` o `wait()`. La tarea se ejecuta entonces en el mismo hilo que realiza la espera.

Por tanto, `std::launch::async` resulta adecuado para aplicar paralelismo funcional entre tareas independientes, mientras que `std::launch::deferred` se comporta como una evaluación diferida sin paralelismo efectivo.

Uso de `std::launch::async`

El parámetro `std::launch::async` se utiliza para garantizar que la tarea se lance inmediatamente en un nuevo hilo. Esto evita que el sistema decida posponer la ejecución y asegura que exista concurrencia real.

En esta práctica, este comportamiento es especialmente importante para ejecutar simultáneamente los algoritmos SRM, ELA y DCT.

Sincronización y recuperación de resultados

La ejecución asíncrona se gestiona mediante objetos `std::future`, que permiten sincronizar tareas y recuperar resultados.

- `wait()`: bloquea la ejecución hasta que la tarea finaliza, sin devolver ningún resultado.
- `get()`: espera a que la tarea termine y devuelve el valor producido por la función.

Es importante destacar que `get()` solo puede invocarse una vez por cada objeto `future`, ya que consume el resultado almacenado.

Ventajas frente a `std::thread`

El uso de `std::async` presenta varias ventajas respecto a `std::thread`:

- Permite obtener el resultado de la tarea directamente mediante `std::future`.
- Simplifica la gestión de hilos, evitando la necesidad de crear y sincronizar manualmente.
- Facilita el tratamiento de excepciones generadas durante la ejecución asíncrona.
- Reduce la complejidad del código al gestionar automáticamente el ciclo de vida del hilo.

En el entorno Linux utilizado en esta práctica, la ejecución asíncrona requiere enlazar la aplicación con la librería POSIX Threads mediante el *flag* `-lpthread`.

4.1.3. `std::vector`

En este apartado se analiza cómo influye el uso de `std::vector` en el rendimiento y en la posibilidad de paralelizar el código dentro de un entorno de memoria compartida.


```
Default initialization: 198ms
Initialization with size: 41ms
```

Figura 2. Resultados de tiempo para dos formas de inicialización de `std::vector`.

Se ejecutó el programa comparando dos formas de inicializar un vector utilizando un tamaño de 10.000.000 de elementos, con el objetivo de observar diferencias de rendimiento.

Los resultados obtenidos fueron los siguientes:

- Inicialización mediante `push_back()`: 198 ms.
- Inicialización con tamaño fijo: 41 ms.

Se observa que el uso de `push_back()` es considerablemente más lento debido a las reasignaciones dinámicas de memoria que realiza el vector al crecer progresivamente. Cada vez que el contenedor supera su capacidad reservada, se necesita asignar un nuevo bloque de memoria de mayor tamaño y copiar los elementos existentes, generando una sobrecarga adicional.

En cambio, al definir el tamaño desde el inicio, la memoria se reserva una única vez, evitando copias innecesarias y permitiendo un acceso directo más eficiente.

Esto demuestra que la estrategia de inicialización del vector tiene un impacto importante en el rendimiento global del programa.

Comparación de métodos de inicialización

En el ejercicio se comparan dos formas distintas de rellenar un vector:

- Inicialización mediante `push_back()`: el vector comienza vacío y los elementos se insertan progresivamente.
- Inicialización con tamaño fijo y acceso por índice: el vector se crea directamente con el tamaño requerido y los valores se asignan mediante `v[i]`.

La segunda opción resulta claramente más eficiente.

Esto ocurre porque `push_back()` puede provocar múltiples realocaciones internas, mientras que un vector preasignado mantiene una estructura de memoria estable y evita costes adicionales de copia.

Problemas potenciales al paralelizar

Desde el punto de vista de la paralelización, existe una diferencia importante entre ambos enfoques:

- Uso de `push_back()`: no es seguro en entornos paralelos (*thread-unsafe*). Si varios hilos intentan insertar elementos simultáneamente sobre el mismo

vector, pueden producirse condiciones de carrera al modificarse de forma concurrente el tamaño, la capacidad y los punteros internos del contenedor.

- **Acceso por índice ($v[i] = \text{valor}$):** este enfoque sí resulta adecuado para paralelismo. Cada hilo trabaja sobre posiciones distintas del vector, manteniendo independencia de datos y evitando conflictos entre accesos concurrentes.

Por tanto, en aplicaciones paralelas resulta recomendable utilizar vectores preasignados y acceso por índice, ya que proporcionan mayor rendimiento y eliminan riesgos asociados a sincronización y corrupción de memoria.

4.2. Análisis del problema

En esta fase inicial se estudió la estructura global del sistema de análisis forense con el objetivo de determinar la viabilidad de la paralelización dentro de una arquitectura de memoria compartida centralizada. En este tipo de sistemas, todos los núcleos del procesador acceden al mismo espacio de direcciones, permitiendo que los datos sean visibles para todos los hilos sin necesidad de duplicación en memoria.

Se analizó el flujo completo de ejecución del programa, identificando las distintas etapas de procesamiento aplicadas sobre la imagen de entrada.

En primer lugar, se identificaron las tres funciones principales del sistema:

- **SRM (Steganalysis Rich Model):** orientado al análisis del patrón de ruido mediante una convolución con un kernel específico aplicado sobre la imagen original.
- **ELA (Error Level Analysis):** encargado de detectar inconsistencias derivadas de la recompresión JPEG y de la comparación entre la imagen original y la recomprimida.
- **DCT (Discrete Cosine Transform):** utilizado para analizar la distribución de energía en las frecuencias de la imagen, permitiendo detectar anomalías especialmente en altas frecuencias.

El algoritmo SRM permite extraer características relacionadas con el ruido generado por el sensor de la cámara. ELA compara la imagen original con una versión recomprimida para detectar posibles zonas manipuladas. Por su parte, DCT estudia la distribución frecuencial de la imagen, siendo especialmente útil para identificar alteraciones producidas mediante herramientas de inteligencia artificial.

Posteriormente, se elaboró un diagrama de dependencias para representar la secuencia de ejecución del sistema.

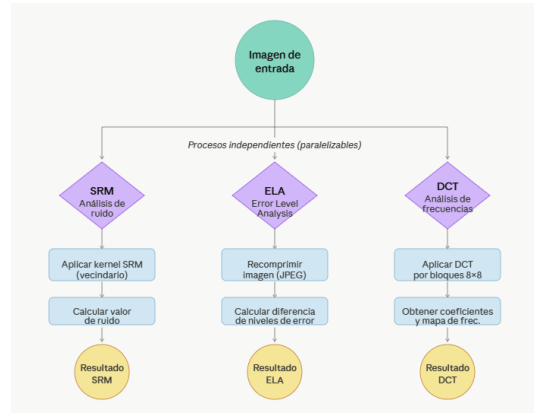


Figura 3. Esquema de dependencias entre los procesos SRM, ELA y DCT.

Este análisis permitió comprobar que SRM, ELA y DCT no presentan dependencias directas entre sí, ya que todos operan sobre la misma imagen de entrada sin requerir resultados intermedios generados por otros procesos.

Por ello, resulta posible aplicar paralelismo funcional ejecutando estos algoritmos de forma concurrente.

Finalmente, se definió un modelo híbrido de paralelismo, combinando dos estrategias complementarias:

- **Paralelismo funcional:** mediante `std::async` con política `std::launch::async`, permitiendo ejecutar simultáneamente los distintos análisis.
- **Paralelismo de datos:** mediante OpenMP, distribuyendo el procesamiento interno de píxeles o bloques entre varios hilos.

Este enfoque permite aprovechar eficientemente la arquitectura multiprocesador de memoria compartida, minimizando copias innecesarias y maximizando el aprovechamiento de los recursos hardware disponibles.

4.3. Análisis de procesos

Antes de implementar la solución paralela, se realizó un estudio detallado del comportamiento secuencial de cada algoritmo con el fin de identificar las regiones de mayor coste computacional. Este análisis permitió comprender la estructura interna de los distintos procesos y localizar las partes más adecuadas para aplicar paralelismo.

Para ello, se elaboraron diagramas de flujo correspondientes a los algoritmos SRM, ELA y DCT, representando la lógica de ejecución y el recorrido de datos

dentro de cada etapa de procesamiento. Estos diagramas facilitaron la identificación de los bucles más intensivos desde el punto de vista computacional, especialmente aquellos relacionados con el recorrido de píxeles y el procesamiento por bloques.

A partir de este análisis, se determinaron los puntos candidatos para la paralelización, priorizando aquellas regiones donde las operaciones podían ejecutarse de forma independiente y sin dependencias de datos.

1. Análisis de ruido (SRM – Steganalysis Rich Model)

Este proceso busca detectar inconsistencias en el patrón de ruido generado por el sensor de la cámara, especialmente útiles para identificar manipulaciones locales sobre la imagen.

- **Lógica interna:** el algoritmo aplica una convolución utilizando el kernel SRM sobre la imagen original.
- **Bucle de mayor carga:** consiste en un doble bucle anidado que recorre cada píxel de la imagen por filas y columnas. Para cada posición, se realizan operaciones matemáticas basadas en los valores de los píxeles vecinos definidos por el kernel.
- **Punto paralelizable:** dado que el cálculo de cada píxel depende únicamente de la lectura de valores vecinos de la imagen original y no de resultados previamente calculados, este doble bucle constituye un candidato ideal para paralelización mediante OpenMP.

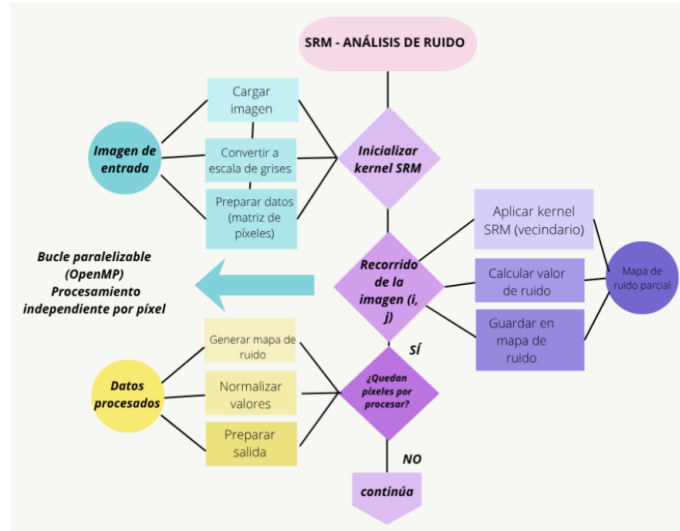


Figura 4. Esquema del análisis de ruido (SRM).

2. Análisis de nivel de error (ELA – Error Level Analysis)

Este algoritmo detecta inconsistencias en la compresión JPEG de una imagen, permitiendo identificar regiones que podrían haber sido modificadas o insertadas desde otras fuentes con diferente nivel de calidad.

■ Lógica interna:

1. Se toma la imagen original y se genera una versión recomprimida utilizando un nivel de calidad JPEG determinado.
 2. Se realiza una resta píxel a píxel entre la imagen original y la imagen recomprimida.
 3. Las diferencias obtenidas se amplifican visualmente para resaltar posibles inconsistencias de compresión.
- **Bucle de mayor carga:** el mayor coste computacional se encuentra en el doble bucle que recorre la imagen para comparar cada píxel de la versión original con el correspondiente de la imagen recomprimida.
 - **Punto paralelizable:** dado que la comparación de cada píxel es independiente del resto, el recorrido de la imagen constituye un candidato ideal para paralelización mediante OpenMP. En un entorno de memoria compartida, ambas imágenes pueden declararse como variables compartidas, permitiendo que todos los hilos accedan simultáneamente a los datos sin necesidad de copias adicionales.

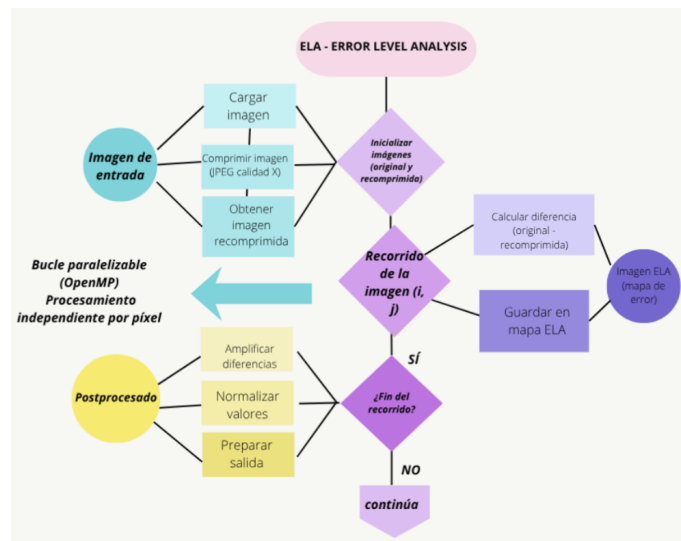


Figura 5. Esquema del análisis de nivel de error (ELA).

3. Análisis de frecuencias (DCT – Discrete Cosine Transform)

Este algoritmo se utiliza para analizar la distribución de frecuencias presentes en la imagen, siendo especialmente útil para detectar huellas asociadas a manipulaciones generadas mediante inteligencia artificial.

- **Lógica interna:** la DCT descompone la imagen en coeficientes de frecuencia utilizando funciones coseno. Las bajas frecuencias, asociadas a estructuras generales de la imagen, se sitúan en la esquina superior izquierda de la transformada, mientras que las altas frecuencias se localizan en la esquina inferior derecha.
- **Bucle de mayor carga:** el proceso requiere dividir la imagen en bloques y aplicar la transformada a cada uno de ellos. Este procedimiento implica un elevado coste computacional, especialmente en imágenes de gran resolución.
- **Punto paralelizable:** cada bloque de la imagen puede procesarse de manera independiente, ya que no existen dependencias entre bloques distintos. Esto permite aplicar paralelismo de datos mediante OpenMP, distribuyendo el cálculo de los coeficientes entre varios hilos de ejecución.

Además, este algoritmo resulta especialmente adecuado para combinarse con paralelismo funcional, ya que puede ejecutarse simultáneamente junto a SRM y ELA mediante `std::async`.

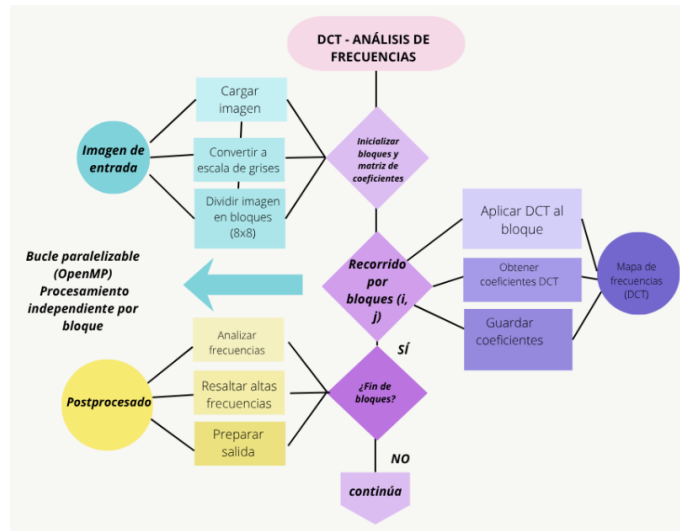


Figura 6. Esquema del análisis de frecuencias (DCT).

4.3.1. Identificación de puntos paralelizables específicos

En los algoritmos analizados, los principales puntos paralelizables se encuentran en los bucles encargados de recorrer la imagen, ya sea a nivel de píxel o de bloque. Estas estructuras iterativas concentran la mayor carga computacional y, en los casos estudiados, no presentan dependencias de datos entre iteraciones, lo que permite una descomposición eficiente del problema.

En los algoritmos SRM y ELA, el punto clave se localiza en el doble bucle que recorre la imagen por filas y columnas. En SRM, este recorrido se utiliza para aplicar el kernel de convolución sobre cada píxel, mientras que en ELA se realiza la comparación entre la imagen original y la recomprimida.

Dado que el cálculo de cada píxel depende únicamente de la lectura de valores de entrada y no modifica resultados intermedios utilizados por otras iteraciones, estos bucles pueden paralelizarse mediante OpenMP utilizando directivas como `#pragma omp parallel for`.

En ambos casos, la imagen puede declararse como variable compartida (`shared`), mientras que los índices de iteración se consideran privados (`private`), garantizando independencia entre hilos y evitando condiciones de carrera.

En el algoritmo DCT, la paralelización se aplica sobre el recorrido de bloques de la imagen. La transformada se calcula sobre regiones independientes, normalmente de tamaño fijo (por ejemplo, bloques de 8x8 píxeles), permitiendo distribuir el trabajo entre varios hilos sin solapamiento de datos.

Cada bloque puede procesarse de forma independiente, ya que la transformación de un bloque no requiere información de otros bloques vecinos. Esto convierte a DCT en uno de los algoritmos más adecuados para paralelismo de datos.

Por ejemplo, en SRM y ELA se trabaja principalmente a nivel de píxel, mientras que en DCT el procesamiento se realiza por bloques.

Pseudocódigo SRM/ELA

```
PARA cada i desde 0 hasta height - 1
  PARA cada j desde 0 hasta width - 1
    resultado[i][j] <- procesar_pixel(i, j)
  FIN
FIN
```

Pseudocódigo DCT

```
PARA cada i desde 0 hasta height - 1 con salto 8
  PARA cada j desde 0 hasta width - 1 con salto 8
    bloque <- obtener_bloque(i, j)
    coeficientes <- aplicar_DCT(bloque)
    guardar_coeficientes_en_resultado
```

FIN
FIN

4.3.2. Medición de tiempos de la versión secuencial

Se midieron los tiempos de ejecución de los distintos procesos en su versión secuencial utilizando un único hilo de ejecución. Estas medidas permiten establecer una referencia inicial que posteriormente se utilizará para calcular el *speed-up* y evaluar el impacto del paralelismo.

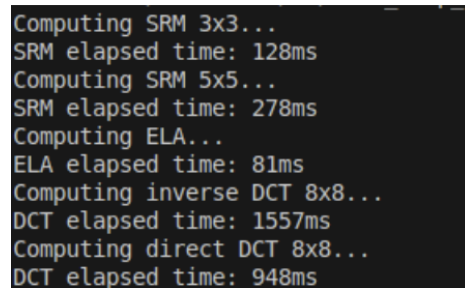
Los procesos analizados corresponden a los tres algoritmos principales del sistema: SRM (con kernels de 3x3 y 5x5), ELA y DCT (en sus variantes directa e inversa). Cada algoritmo se ejecuta de forma independiente, permitiendo medir su tiempo de ejecución de manera aislada.

La medición se realizó mediante la librería `<chrono>` de C++, utilizando `std::chrono::steady_clock` por tratarse de un reloj monótono adecuado para medir intervalos temporales.

Código:

```
auto begin = std::chrono::steady_clock::now();
...
auto end = std::chrono::steady_clock::now();
```

La diferencia entre ambos instantes proporciona el tiempo total de ejecución del algoritmo evaluado.



```
Computing SRM 3x3...
SRM elapsed time: 128ms
Computing SRM 5x5...
SRM elapsed time: 278ms
Computing ELA...
ELA elapsed time: 81ms
Computing inverse DCT 8x8...
DCT elapsed time: 1557ms
Computing direct DCT 8x8...
DCT elapsed time: 948ms
```

Figura 7. Tiempos obtenidos en una ejecución de la versión secuencial.

A partir de los resultados obtenidos, se observa que el algoritmo DCT presenta el mayor coste computacional, especialmente en su versión inversa, debido al procesamiento intensivo por bloques y al elevado número de operaciones matemáticas necesarias.

Por el contrario, ELA presenta los tiempos más reducidos, ya que consiste principalmente en operaciones de comparación y resta entre píxeles.

En el caso de SRM, se aprecia un aumento del tiempo de ejecución al incrementar el tamaño del kernel de convolución, debido al mayor número de accesos a memoria y operaciones necesarias por cada píxel.

Estos resultados constituyen la referencia base utilizada posteriormente para comparar el rendimiento de la versión paralela.

4.3.3. Características del sistema (hardware)

Para contextualizar los resultados de rendimiento obtenidos, se realizó una caracterización del sistema hardware utilizado mediante las herramientas `lscpu` y `cat /proc/cpuinfo`.

Características	Valor
Procesador	Intel i7-13620H
Núcleos	10
Hilos	16
Frecuencia máxima	4.9 GHz
Caché L3	24 MB
Arquitectura	x86_64

Cuadro 1. Características del sistema utilizado en las pruebas.

El programa se ejecutó en un equipo HP Victus Gaming equipado con un procesador Intel Core i7-13620H de 13^a generación, basado en arquitectura x86_64.

Se trata de una arquitectura multiprocesador de memoria compartida centralizada, donde todos los hilos acceden al mismo espacio de direcciones. Esta característica permite utilizar variables compartidas sin necesidad de duplicar datos entre hilos, aspecto especialmente importante al trabajar con imágenes de gran tamaño.

El procesador dispone de 10 núcleos físicos y 16 hilos de ejecución. La presencia del flag *Hyper-Threading* indica que algunos núcleos pueden ejecutar dos hilos simultáneamente, mejorando el aprovechamiento de las unidades funcionales del procesador.

La memoria caché L3 de 24 MB resulta especialmente relevante para algoritmos intensivos en acceso a memoria, como SRM y DCT, ya que reduce la latencia asociada al acceso repetitivo a datos.

Desde el punto de vista del paralelismo, el sistema permite ejecutar varias tareas de forma simultánea. Sin embargo, al compartir recursos internos dentro de un mismo núcleo físico, el aumento del rendimiento no siempre crece de manera lineal con el número de hilos.

Por ello, es esperable observar una mejora significativa del rendimiento hasta cierto umbral, a partir del cual el beneficio adicional disminuye debido a la competición por recursos hardware compartidos.

4.4. Diseño de la solución paralela

Esta sección describe el desarrollo de la arquitectura paralela implementada para mejorar el rendimiento del sistema de análisis forense. La solución se diseñó para aprovechar una arquitectura multiprocesador de memoria compartida, donde todos los hilos pueden acceder al mismo espacio de direcciones.

4.4.1. Diseño de la solución paralela

Para optimizar el tiempo de ejecución de la aplicación, se implementó una estrategia híbrida basada en dos niveles de paralelismo complementarios: paralelismo de datos mediante OpenMP y paralelismo funcional mediante `std::async`.

Paralelismo de datos mediante OpenMP

OpenMP se utilizó para paralelizar los bucles internos de mayor coste computacional dentro de cada algoritmo.

En los algoritmos SRM y ELA, la paralelización se aplicó sobre los dobles bucles encargados de recorrer la imagen píxel a píxel. Dado que cada iteración trabaja sobre posiciones independientes y no existen dependencias entre resultados, el procesamiento puede distribuirse entre varios hilos utilizando directivas como:

```
#pragma omp parallel for
```

Las estructuras de datos correspondientes a la imagen se declararon como variables compartidas (`shared`), mientras que los índices de iteración se trataron como variables privadas (`private`) para evitar conflictos entre hilos.

En el algoritmo DCT, la paralelización se realizó sobre el recorrido de bloques de la imagen. Cada bloque puede transformarse de forma independiente, permitiendo repartir el cálculo de coeficientes frecuenciales entre distintos hilos.

Este enfoque aprovecha la arquitectura de memoria compartida, evitando duplicaciones innecesarias de datos y reduciendo el coste asociado al acceso a memoria.

Paralelismo funcional mediante `std::async`

A un nivel superior, se empleó programación asíncrona para ejecutar simultáneamente los algoritmos SRM, ELA y DCT.

Para ello, se utilizó `std::async` junto con la política `std::launch::async`, garantizando que cada tarea se ejecute en un hilo independiente y evitando la evaluación diferida.

```
std::future<Result> tarea =  
std::async(std::launch::async, funcion);
```

Cada llamada devuelve un objeto `std::future`, que permite sincronizar la ejecución y recuperar el resultado correspondiente mediante el método `get()`.

Esta estrategia permite ejecutar los distintos análisis de forma concurrente, aprovechando que no existen dependencias directas entre ellos.

Combinación de estrategias

La solución implementada combina dos niveles de paralelismo:

- **Paralelismo de datos:** aplicado dentro de cada algoritmo mediante OpenMP, repartiendo el procesamiento de píxeles o bloques entre varios hilos.
- **Paralelismo funcional:** aplicado entre algoritmos mediante `std::async`, ejecutando SRM, ELA y DCT de manera concurrente.

Esta combinación permite aprovechar tanto los núcleos físicos como los hilos lógicos disponibles en el procesador, maximizando el uso del hardware.

Además, el uso conjunto de OpenMP y programación asíncrona permite explotar la jerarquía del paralelismo: tareas independientes a alto nivel y procesamiento paralelo interno dentro de cada tarea.

Como resultado, se obtiene una solución más eficiente que reduce el tiempo total de ejecución respecto a la versión secuencial.

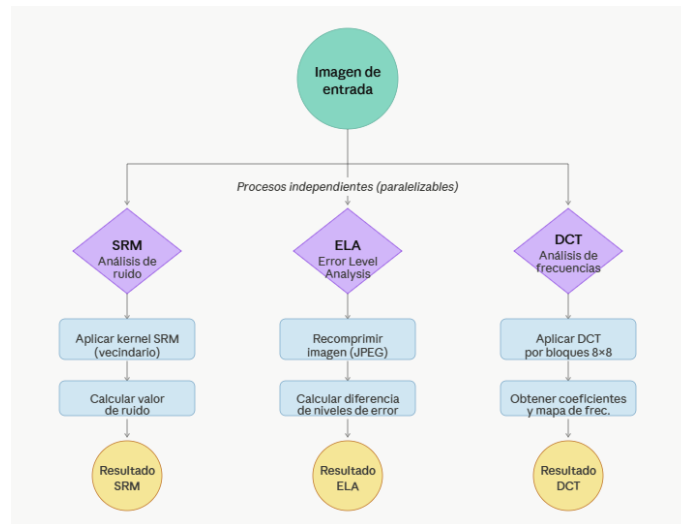
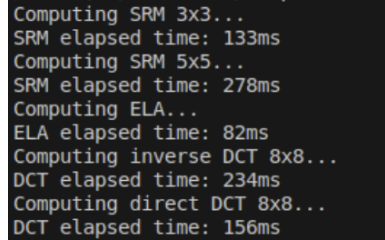


Figura 8. Grafo de paralelismo asociado a la combinación de estrategias empleadas.

4.4.2. Implementación y cambios realizados

Para implementar la versión paralela del programa, se realizaron modificaciones sobre el código secuencial original con el objetivo de aprovechar el paralelismo disponible en una arquitectura de memoria compartida. Estas modificaciones se centraron en dos niveles: paralelismo de datos mediante OpenMP y paralelismo funcional mediante `std::async`.



```
Computing SRM 3x3...
SRM elapsed time: 133ms
Computing SRM 5x5...
SRM elapsed time: 278ms
Computing ELA...
ELA elapsed time: 82ms
Computing inverse DCT 8x8...
DCT elapsed time: 234ms
Computing direct DCT 8x8...
DCT elapsed time: 156ms
```

Figura 9. Esquema general de la solución paralela implementada.

Paralelismo de datos con OpenMP

Se introdujo paralelización en el algoritmo DCT, identificado previamente como la parte de mayor coste computacional del sistema. Este algoritmo divide la imagen en bloques independientes, lo que permite procesarlos en paralelo sin dependencias de datos entre ellos.

Para ello, se utilizó la directiva `#pragma omp parallel for` aplicada sobre el bucle que recorre los bloques de la imagen:

```
#pragma omp parallel for
for(int i = 0; i < blocks.size(); i++){
    float **dctBlock = dct::create_matrix(block_size, block_size);
    dct::direct(dctBlock, blocks[i], 0);
    if (invert) {
        for(int k = 0; k < blocks[i].size/2; k++)
            for(int l = 0; l < blocks[i].size/2; l++)
                dctBlock[k][l] = 0.0;
        dct::inverse(blocks[i], dctBlock, 0, 0.0, 255.);
    } else {
        dct::assign(dctBlock, blocks[i], 0);
    }
    dct::delete_matrix(dctBlock);
}
```

Esta implementación permite distribuir el procesamiento de bloques entre varios hilos. Cada hilo trabaja sobre un subconjunto distinto de bloques, evitando interferencias entre iteraciones y reduciendo significativamente el tiempo de ejecución.

En este caso, la estructura que almacena los bloques puede considerarse compartida entre los hilos, mientras que las matrices temporales utilizadas para el cálculo se crean localmente dentro de cada iteración, evitando conflictos de escritura.

Paralelismo funcional con `std::async`

Además del paralelismo interno, se implementó paralelismo funcional mediante `std::async`. En la versión secuencial, los algoritmos SRM, ELA y DCT se ejecutaban de forma consecutiva. En la versión paralela, estos procesos se lanzan de manera concurrente utilizando la política `std::launch::async`.

```
auto srm3 = std::async(std::launch::async, compute_srm, image, 3);
auto srm5 = std::async(std::launch::async, compute_srm, image, 5);
auto ela = std::async(std::launch::async, compute_ela, image, 90);
auto dct_inv = std::async(std::launch::async, compute_dct, image, block_size, true);
auto dct_dir = std::async(std::launch::async, compute_dct, image, block_size, false);

save_to_file("srm_kernel_3x3.png", srm3.get());
save_to_file("srm_kernel_5x5.png", srm5.get());
save_to_file("ela.png", ela.get());
save_to_file("dct_invert.png", dct_inv.get());
save_to_file("dct_direct.png", dct_dir.get());
```

El uso de `std::launch::async` garantiza que cada tarea se ejecute inmediatamente en un hilo independiente, evitando el comportamiento diferido asociado a `std::launch::deferred`.

La sincronización se realiza mediante los objetos `std::future` y el método `get()`, que bloquea la ejecución hasta que el resultado correspondiente está disponible.

Resolución de conflictos en ELA

Durante la ejecución concurrente se detectó un posible conflicto en el algoritmo ELA. La versión original utilizaba un único archivo temporal para almacenar la imagen recomprimida, lo que podía provocar condiciones de carrera cuando varias tareas intentaban acceder simultáneamente al mismo recurso.

Para evitar este problema, se implementó un sistema de nombres dinámicos para los archivos temporales:

```
std::string temp_name = "_temp_" + std::to_string(rand()) + ".jpg";

save_to_file(temp_name, grayscale, quality);
Image<float> compressed = load_from_file(temp_name).convert<float>();
```

Este cambio garantiza que cada tarea utilice un archivo independiente, evitando accesos simultáneos y permitiendo una ejecución segura en paralelo.

Consideraciones adicionales

En los algoritmos SRM y ELA no se aplicó paralelización interna completa debido a limitaciones derivadas de funciones de librería y operaciones de entrada/salida. No

obstante, ambos procesos se benefician del paralelismo funcional al ejecutarse de forma concurrente con el resto de algoritmos.

En conjunto, las modificaciones realizadas permiten aprovechar simultáneamente el paralelismo de tareas y el paralelismo de datos, reduciendo el tiempo total de ejecución y mejorando el aprovechamiento de los recursos hardware disponibles.

4.4.3. Evaluación del rendimiento y *speed-up*

Para evaluar el impacto de la paralelización, se compararon los tiempos de ejecución obtenidos en la versión secuencial y en la versión paralela del programa. A partir de estas mediciones, se calcularon distintas métricas de rendimiento que permiten cuantificar la mejora derivada del uso de paralelismo en una arquitectura de memoria compartida.

Métricas de rendimiento

La principal métrica utilizada es el *speed-up*, que expresa la relación entre el tiempo de ejecución secuencial y el tiempo de ejecución paralelo utilizando p hilos.

$$S(p) = \frac{T_{sec}}{T_{par}(p)} \quad (1)$$

donde T_{sec} representa el tiempo de ejecución de la versión secuencial y $T_{par}(p)$ el tiempo correspondiente a la versión paralela utilizando p hilos.

Esta métrica permite medir cuántas veces se acelera el programa al aplicar paralelismo, siendo especialmente útil para analizar la escalabilidad y el aprovechamiento del hardware.

Además del *speed-up*, se consideró la eficiencia, definida como:

$$E(p) = \frac{S(p)}{p} \quad (2)$$

La eficiencia indica el grado de aprovechamiento de los hilos disponibles. Valores cercanos a 1 representan un uso ideal de los recursos, mientras que valores inferiores reflejan pérdidas asociadas a sincronización, acceso a memoria o sobrecoste de gestión.

Ley de Amdahl

Para estimar la aceleración máxima teórica, se empleó la Ley de Amdahl, que establece el límite de mejora alcanzable según la fracción paralelizable del programa.

$$S_{max} = \frac{1}{(1 - P) + \frac{P}{N}} \quad (3)$$

donde P representa la proporción paralelizable del programa y N el número de hilos utilizados.

Esta ley muestra que el rendimiento global siempre está condicionado por la parte secuencial del programa. Aunque aumente el número de hilos, la aceleración queda limitada por aquellas regiones que no pueden ejecutarse en paralelo.

Resultados obtenidos

La Tabla 2 muestra la comparación entre los tiempos secuenciales y paralelos obtenidos para cada algoritmo.

Cuadro 2. Comparación de tiempos y *speed-up* por algoritmo.

Algoritmo	Secuencial (ms)	Paralelo (ms)	<i>Speed-up</i>
SRM 3x3	134	133	1.0
SRM 5x5	278	278	1.0
ELA	81	82	1.0
DCT inversa	1555	234	6.6
DCT directa	946	156	6.0

Interpretación de resultados

Los resultados muestran que la paralelización produce una mejora significativa en el algoritmo DCT, alcanzando un *speed-up* superior a 6 tanto en la versión directa como en la inversa.

Esta mejora se debe a la aplicación de OpenMP sobre el procesamiento por bloques. Cada bloque de la imagen puede procesarse de forma independiente, lo que permite repartir el trabajo entre varios hilos y aprovechar los recursos del procesador.

No obstante, el *speed-up* obtenido no alcanza el valor ideal asociado al número total de hilos disponibles. Esto se explica por factores como el sobrecoste de creación y sincronización de hilos, el acceso concurrente a memoria y la competencia por recursos compartidos como la caché.

En los algoritmos SRM y ELA no se observan mejoras significativas en el tiempo de ejecución. Esto se debe a que no se implementó paralelización interna completa en estos procesos. En el caso de SRM, parte del procesamiento depende de funciones de librería externas, mientras que ELA incorpora operaciones de entrada/salida que dificultan la paralelización eficiente.

Aun así, ambos algoritmos se benefician indirectamente del paralelismo funcional introducido mediante `std::async`, ya que se ejecutan de forma concurrente con DCT, reduciendo el tiempo total de la aplicación.

4.4.4. Discusión de resultados y conclusiones

En este apartado se analizan los resultados obtenidos y se responde a las cuestiones principales relacionadas con la eficiencia del paralelismo aplicado al sistema de análisis forense.

¿Se obtiene un mejor rendimiento paralelizando todas las partes posibles?

No necesariamente. La paralelización no garantiza una mejora automática del rendimiento en todas las regiones del programa. Según la Ley de Amdahl, la aceleración máxima alcanzable está limitada por la fracción secuencial del código.

En este trabajo, la mejora significativa se ha concentrado en el algoritmo DCT, ya que su estructura basada en bloques independientes permite aplicar paralelismo de datos de forma eficiente. Sin embargo, otros algoritmos como SRM y ELA no presentan una mejora apreciable, debido a que contienen operaciones dependientes de funciones de librería o acceso a disco.

Esto demuestra que no todas las partes del programa son adecuadas para paralelizarse y que resulta más eficiente centrarse en aquellas regiones donde el coste computacional justifica el uso de hilos.

¿Se degrada el rendimiento al paralelizar ciertas partes?

Sí. En determinados casos, la paralelización puede producir un empeoramiento del rendimiento si se aplica sobre tareas que no presentan suficiente carga computacional o si existen conflictos entre hilos.

Por ejemplo, durante el desarrollo se observó que el algoritmo ELA podía generar problemas cuando varios hilos accedían simultáneamente a un mismo archivo temporal. Este comportamiento provocaba condiciones de carrera y reducía la estabilidad del sistema.

Además, cuando las tareas son relativamente pequeñas o rápidas, el coste asociado a crear, sincronizar y finalizar hilos puede superar el beneficio obtenido mediante paralelismo.

¿A qué se debe la posible degradación del rendimiento?

La degradación se debe principalmente al sobre coste asociado a la gestión del paralelismo.

La creación de hilos, la sincronización entre tareas y el acceso concurrente a memoria compartida introducen un coste adicional que no existe en la versión secuencial. Además, cuando varios hilos comparten recursos hardware, como memoria caché o ancho de banda de memoria, pueden aparecer cuellos de botella.

En arquitecturas con tecnología Hyper-Threading, varios hilos lógicos comparten recursos internos del mismo núcleo, lo que puede limitar la ganancia real al aumentar el número de hilos.

Por ello, el paralelismo resulta más rentable cuando el trabajo computacional es suficientemente grande como para compensar este sobre coste.

¿Cuál es la mejor implementación y estrategia de paralelización final?

La estrategia más eficiente ha sido aplicar un modelo híbrido que combina paralelismo funcional y paralelismo de datos.

Por un lado, se utilizó `std::async` con la política `std::launch::async` para ejecutar de forma concurrente los distintos algoritmos del sistema. Esto permite solapar tareas independientes y reducir el tiempo total de ejecución.

Por otro lado, se aplicó OpenMP en el algoritmo DCT, donde la independencia entre bloques hace posible distribuir el trabajo entre varios hilos sin dependencias de datos.

Esta combinación permite aprovechar tanto el paralelismo a nivel de tareas como el paralelismo interno dentro de los algoritmos más costosos.

En consecuencia, la estrategia final no consiste en paralelizar todo el programa, sino en identificar aquellas regiones donde el beneficio es real y donde el coste de coordinación no supera la ganancia obtenida.

5. Resultados

En esta sección se presentan los resultados obtenidos tras ejecutar la versión secuencial y la versión paralela del sistema de análisis forense. Las pruebas se realizaron sobre una arquitectura de memoria compartida equipada con un procesador Intel i7-13620H, utilizando hasta 16 hilos de ejecución.

5.1. Tiempos secuenciales

Como referencia inicial, se midieron los tiempos de ejecución de la versión secuencial utilizando un único hilo. Estos resultados permiten establecer una línea base para comparar posteriormente el efecto de la paralelización.

Algoritmo	Tiempo (ms)
SRM 3x3	134
SRM 5x5	278
ELA	81
DCT inversa	1555
DCT directa	946

Cuadro 3. Tiempos de ejecución de la versión secuencial.

Se observa que el algoritmo DCT concentra la mayor carga computacional del sistema, especialmente en su versión inversa. Esto confirma que el procesamiento frecuencial representa la región más costosa del programa.

5.2. Tiempos paralelos

Posteriormente, se ejecutó la versión paralela utilizando 16 hilos de ejecución. Los tiempos obtenidos se muestran en la Tabla 4.

Algoritmo	Tiempo (ms)
SRM 3x3	133
SRM 5x5	278
ELA	82
DCT inversa	234
DCT directa	156

Cuadro 4. Tiempos de ejecución de la versión paralela.

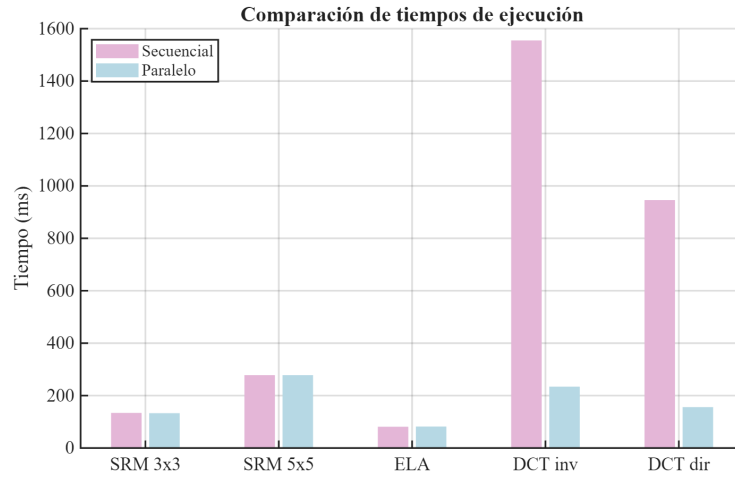


Figura 10. Representación gráfica de los tiempos de ejecución paralelos.

Los resultados muestran una reducción significativa del tiempo en el algoritmo DCT, mientras que SRM y ELA mantienen tiempos similares a los de la versión secuencial.

5.3. *Speed-up*

Para cuantificar la mejora de rendimiento, se calculó el *speed-up* comparando los tiempos secuenciales y paralelos.

Algoritmo	Secuencial (ms)	Paralelo (ms)	<i>Speed-up</i>
SRM 3x3	134	133	1.0
SRM 5x5	278	278	1.0
ELA	81	82	1.0
DCT inversa	1555	234	6.6
DCT directa	946	156	6.0

Cuadro 5. Comparación de tiempos y *speed-up* por algoritmo.

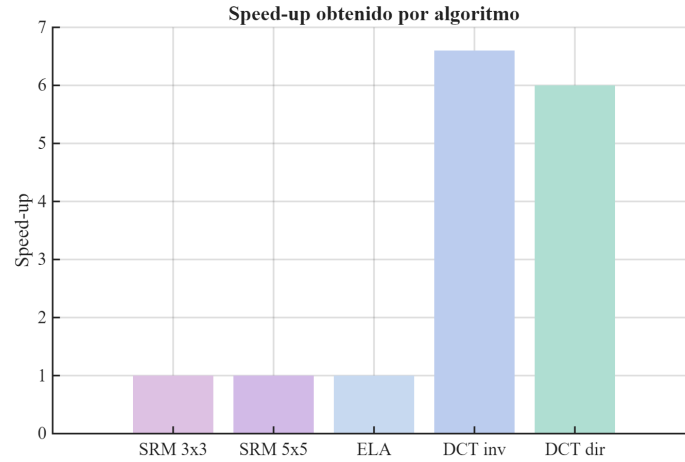


Figura 11. Representación gráfica del *speed-up* obtenido por algoritmo.

El *speed-up* obtenido en el algoritmo DCT demuestra una mejora notable derivada de la paralelización. Esto se debe a que el procesamiento por bloques permite distribuir el trabajo entre varios hilos sin introducir dependencias de datos.

Por el contrario, los algoritmos SRM y ELA muestran valores cercanos a 1, indicando que no existe una mejora significativa. Esto confirma que la paralelización resulta efectiva principalmente cuando el coste computacional es elevado y las operaciones son independientes.

5.4. Eficiencia

La eficiencia permite evaluar el grado de aprovechamiento de los hilos utilizados. Esta métrica se calcula como la relación entre el *speed-up* y el número de hilos empleados.

Algoritmo	<i>Speed-up</i>	Nº hilos	Eficiencia
DCT inversa	6.6	16	0.41
DCT directa	6.0	16	0.38

Cuadro 6. Eficiencia obtenida en los algoritmos DCT paralelizados.

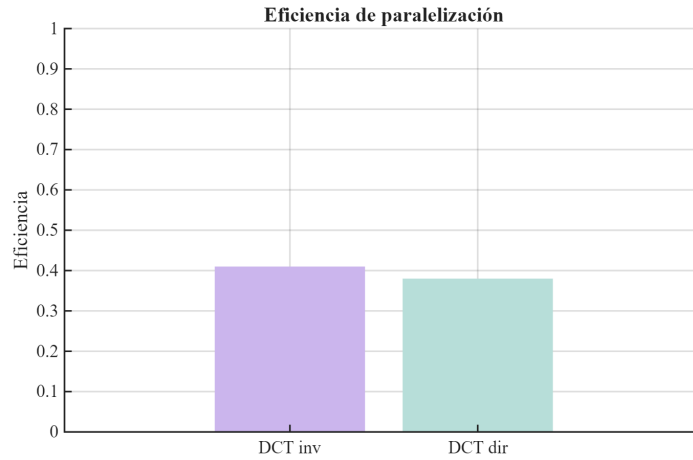


Figura 12. Representación gráfica de la eficiencia obtenida en los algoritmos DCT paralelizados.

Los valores de eficiencia muestran que aproximadamente un 40 % del potencial de paralelismo disponible se aprovecha de forma efectiva.

Esta diferencia respecto al rendimiento ideal se debe al sobrecoste asociado a la creación y sincronización de hilos, así como a la competencia por recursos compartidos del sistema, como la memoria caché y el ancho de banda de memoria.

Además, el uso de tecnología Hyper-Threading implica que varios hilos lógicos comparten recursos internos del procesador, lo que limita el escalado lineal cuando aumenta el número de hilos utilizados.

5.5. Análisis de escalabilidad

Con el objetivo de estudiar el comportamiento del sistema paralelo frente al aumento de recursos computacionales y del tamaño del problema, se realizó un análisis de escalabilidad centrado en el algoritmo DCT. Este algoritmo fue seleccionado debido a que es el único que presenta mejoras significativas tras la paralelización. En los algoritmos SRM y ELA, los tiempos permanecen prácticamente constantes, por lo que su análisis no aporta información relevante desde el punto de vista del rendimiento paralelo.

5.5.1. Escalabilidad respecto al número de hilos

Para analizar la influencia del número de hilos, se ejecutó el programa utilizando distintos valores de paralelismo: 1, 2, 4, 8 y 16 hilos. Para ello, se utilizó la variable de entorno `OMP_NUM_THREADS`, que permite indicar el número de hilos empleados por OpenMP durante la ejecución.

Antes de cada prueba, se configuró el número de hilos desde terminal mediante instrucciones como:

```
export OMP_NUM_THREADS=4
./programa imagen.jpg
```

Esto permitió medir cómo evoluciona el tiempo de ejecución al incrementar los recursos disponibles y calcular métricas como el *speed-up* y la eficiencia.

DCT inversa

Nº Hilos	T_{sec} (ms)	T_{par} (ms)	Speed-up	Eficiencia
1	1555	1549	1.00	1.00
2	1552	793	1.96	0.98
4	1556	420	3.71	0.93
8	1554	334	4.65	0.58
16	1553	243	6.39	0.40

Cuadro 7. Escalabilidad de DCT inversa según el número de hilos.

DCT directa

Nº Hilos	T_{sec} (ms)	T_{par} (ms)	Speed-up	Eficiencia
1	942	942	1.00	1.00
2	939	490	1.92	0.96
4	942	264	3.57	0.89
8	941	213	4.42	0.55
16	938	153	6.13	0.38

Cuadro 8. Escalabilidad de DCT directa según el número de hilos.

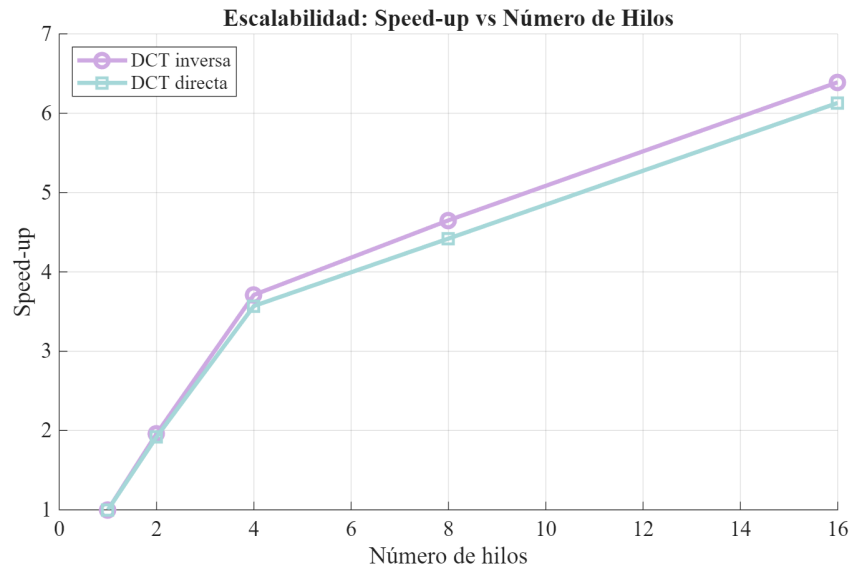


Figura 13. Evolución del *speed-up* de la DCT inversa y directa según el número de hilos.

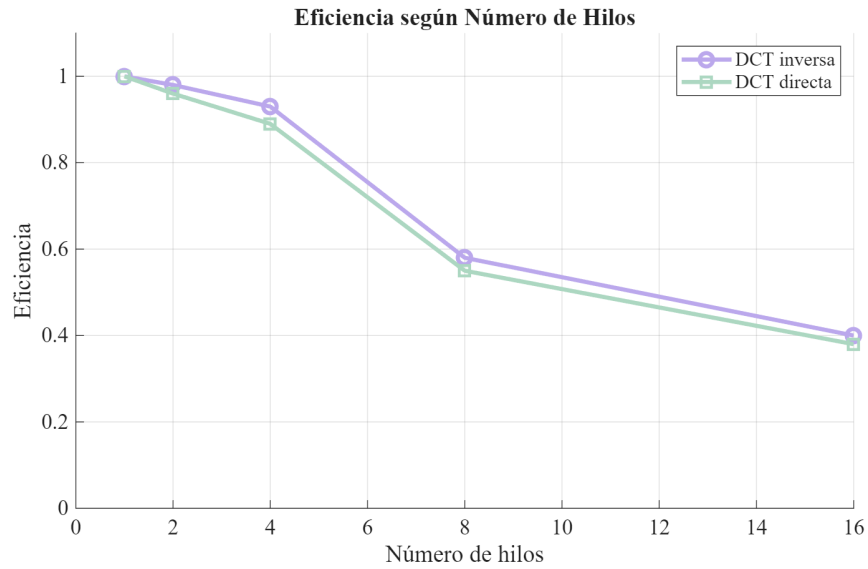


Figura 14. Evolución de la eficiencia de la DCT inversa y directa según el número de hilos.

A partir de los resultados obtenidos, se observa que el *speed-up* aumenta progresivamente al incrementar el número de hilos. Sin embargo, la eficiencia disminuye a medida que crece el paralelismo. Esto ocurre debido al sobrecoste de sincronización, la competencia por recursos compartidos y el efecto de la tecnología *Hyper-Threading*, donde varios hilos comparten recursos físicos dentro del mismo núcleo.

5.5.2. Escalabilidad respecto a la talla del problema

Además del número de hilos, también se analizó cómo influye la resolución de la imagen en el rendimiento paralelo. Para ello, se fijó el número de hilos en 16 y se generaron varias versiones de una misma imagen con diferentes tamaños.

Las imágenes se redimensionaron utilizando la herramienta `convert` de ImageMagick desde terminal:

```
convert imagen.jpg -resize 1024x1024 imagen_1024.jpg
```

Se utilizaron resoluciones de 512x512, 1024x1024, 2048x2048 y 4096x4096 píxeles.

DCT inversa

Resolución	T_{sec} (ms)	T_{par} (ms)	Speed-up
512x512	882	136	6.49
1024x1024	3517	519	6.78
2048x2048	14115	2042	6.91
4096x4096	56394	8088	6.97

Cuadro 9. Escalabilidad de DCT inversa según la resolución de la imagen.

DCT directa

Resolución	T_{sec} (ms)	T_{par} (ms)	Speed-up
512x512	534	92	5.80
1024x1024	2137	386	5.54
2048x2048	8545	1352	6.32
4096x4096	34214	5346	6.40

Cuadro 10. Escalabilidad de DCT directa según la resolución de la imagen.

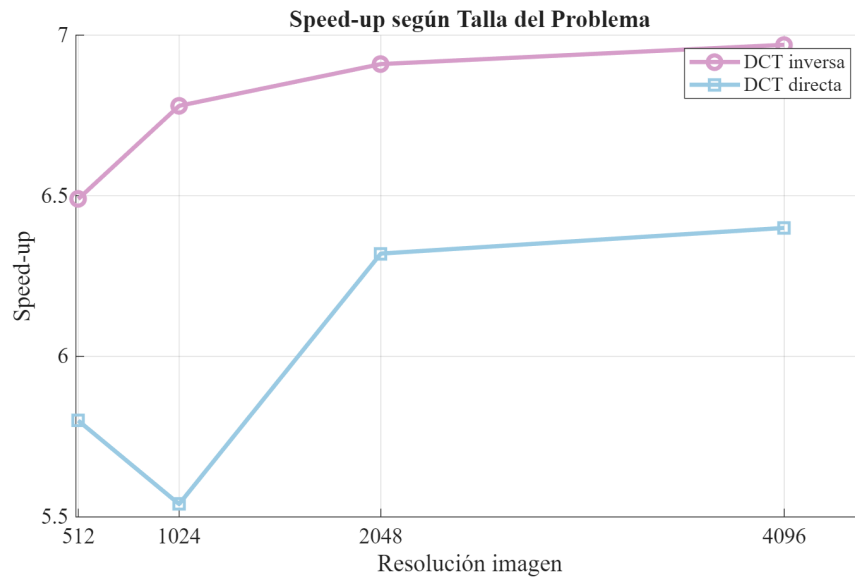


Figura 15. Evolución del *speed-up* de la DCT inversa y directa según la resolución de la imagen.

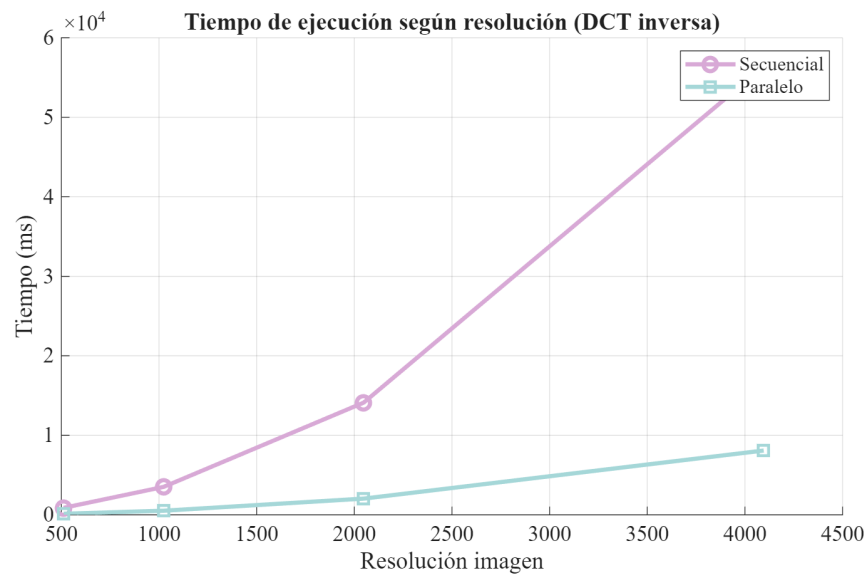


Figura 16. Tiempo de ejecución secuencial y paralelo de la DCT inversa según la resolución de la imagen.

Los resultados muestran que el rendimiento paralelo mejora conforme aumenta la talla del problema. A medida que la resolución de la imagen crece, el coste computacional del algoritmo aumenta y el sobre coste asociado a la creación y sincronización de hilos se vuelve menos relevante. Esto provoca que el *speed-up* se estabilice y se acerque a su máximo rendimiento práctico.

Este comportamiento confirma que el algoritmo DCT presenta una buena escalabilidad, tanto respecto al número de hilos como respecto al tamaño del problema, siendo especialmente adecuado para aprovechar arquitecturas multiprocesador de memoria compartida.

6. Conclusiones

A lo largo de esta práctica se ha comprobado que la paralelización de un programa no implica automáticamente una mejora del rendimiento. Uno de los principales aprendizajes obtenidos es que no todas las partes del código presentan el mismo potencial de paralelización, por lo que resulta fundamental analizar previamente la estructura del problema y las dependencias existentes.

En este trabajo, la mejora de rendimiento se ha concentrado principalmente en el algoritmo DCT, donde se ha conseguido una reducción significativa del tiempo de ejecución mediante OpenMP. Esto ha sido posible gracias a que el procesamiento por bloques permite dividir el trabajo en tareas independientes, adecuadas para el paralelismo de datos. Por el contrario, en algoritmos como SRM y ELA no se han observado mejoras apreciables, debido a que no se han paralelizado internamente o presentan limitaciones asociadas al uso de funciones de librería y operaciones de entrada/salida.

Los resultados obtenidos también permiten validar la Ley de Amdahl, que establece que la aceleración máxima alcanzable está limitada por la fracción secuencial del programa. Aunque el sistema utilizado dispone de múltiples hilos de ejecución gracias a la tecnología Hyper-Threading, el rendimiento global sigue condicionado por aquellas partes del código que no pueden ejecutarse en paralelo.

Además, se ha observado que paralelizar determinadas regiones puede incluso degradar el rendimiento. Esto ocurre debido al sobre coste asociado a la creación, sincronización y gestión de hilos, así como a posibles conflictos derivados del acceso concurrente a recursos compartidos. Por ello, en tareas de baja carga computacional o con dependencias externas, el coste del paralelismo puede superar el beneficio obtenido.

La estrategia más efectiva ha consistido en combinar dos niveles de paralelismo. Por un lado, OpenMP se ha utilizado para acelerar internamente el algoritmo DCT mediante paralelismo de datos. Por otro, `std::async` se ha empleado para ejecutar simultáneamente los distintos análisis (SRM, ELA y DCT), aplicando paralelismo funcional. Para garantizar la ejecución concurrente real, se utilizó la política `std::launch::async`, evitando la evaluación diferida que habría reducido el rendimiento.

Esta solución híbrida ha sido especialmente eficiente gracias a la arquitectura de memoria centralizada del sistema empleado. Todos los hilos comparten el mismo espacio de direcciones, permitiendo acceder a la imagen original sin necesidad de duplicar datos en memoria, lo que reduce el coste asociado al intercambio de información entre tareas.

Finalmente, se ha comprobado que el beneficio del paralelismo aumenta conforme crece la talla del problema. En imágenes de mayor resolución, el coste computacional del procesamiento compensa ampliamente el *overhead* asociado a la gestión de hilos, haciendo que la paralelización resulte más rentable.

En definitiva, la práctica ha permitido comprender que la clave para obtener mejoras reales de rendimiento no consiste en paralelizar todo el programa, sino en identificar correctamente qué regiones presentan suficiente carga computacional e independencia de datos. La combinación de paralelismo funcional y paralelismo de datos ha demostrado ser la estrategia más adecuada para este sistema de análisis forense de imágenes.

Referencias

Referencias

1. OpenMP Architecture Review Board. *OpenMP API Specification*. Disponible en: <https://www.openmp.org/specifications/>
2. Lawrence Livermore National Laboratory. *OpenMP Tutorial*. Disponible en: <https://hpc-tutorials.llnl.gov/openmp/>
3. cppreference. *C++ Concurrency Support Library: std::async and std::future*. Disponible en: <https://en.cppreference.com/w/cpp/thread/async>
4. Kitware Inc. *CMake Documentation*. Disponible en: <https://cmake.org/documentation/>
5. PNG Development Group. *libpng Official Documentation*. Disponible en: <http://www.libpng.org/pub/png/libpng.html>
6. Independent JPEG Group. *libjpeg Documentation*. Disponible en: <http://libjpeg.sourceforge.net/>
7. Michael Kerrisk. *Linux Manual Page: lscpu*. Disponible en: <https://man7.org/linux/man-pages/man1/lscpu.1.html>
8. Wikipedia Contributors. *Amdahl's Law*. Disponible en: https://en.wikipedia.org/wiki/Amdahl%27s_law